

Model-independent meta-fallback system prompts for autonomous LLM agents

The single most impactful change is structural, not textual: use XML tags for section boundaries and Markdown for content within those sections. This hybrid format is the only approach explicitly recommended by Anthropic, OpenAI (for o-series), Google (Gemini 3), and Mistral simultaneously.

(robertodiasduarte) Combined with aggressive instruction positioning (critical rules at the start and re-echoed at the end), structured JSON summarization for context compaction, and conditional reasoning directives that adapt to model type, a ~100-line system prompt can maintain identity persistence across 50+ turn sessions on any major LLM backend. The evidence base spans meta-analyses of 110+ papers, persona drift studies across 9 model families, leaked production prompts from Claude Code (15,000+ tokens assembled dynamically), (GitHub) Cursor, Cline, and Aider, (GitHub) (DeepWiki) and official guidance from every major provider.

Seven findings that should reshape your prompt architecture

1. XML is the universal structural language — Markdown is the universal content language

Research on prompt formatting reveals that **performance can vary up to 40% based on format alone** (measured on GPT-3.5-turbo), though larger models like GPT-4 show more robustness. (arXiv) (robertodiasduarte) The critical finding for model-agnostic design: XML-style tags are the only delimiter format that all major providers explicitly encourage. (robertodiasduarte) Anthropic states Claude has been "specifically tuned to pay special attention to your structure" using XML tags. (Robertodiasduarte) (Anthropic) OpenAI recommends "delimiters like markdown, XML tags, and section titles" for reasoning models. (Robertodiasduarte +2) Google's Gemini 3 documentation states "XML-style tags (e.g., `<context>`, `<task>`) or Markdown headings are effective." (Google AI) Mistral calls them "Readable, Parsable, Familiar." (Mistral AI) (mistral)

Critically, XML outperformed other formats for complex prompts in tests with Llama-3.1-405B and other models, (Robertodiasduarte) while Markdown uses **~15% fewer tokens** than XML for equivalent content. (Robertodiasduarte) (robertodiasduarte) The optimal synthesis: XML tags for structural boundaries between sections, Markdown for readable content within those sections. JSON and YAML consistently underperformed as prompt structure formats across models. Bold and italic emphasis (**`**bold**`**, *`*italic*`*) showed inconsistent effects across models — do not rely on them for behavioral compliance. Numbered lists imply sequence to models; bullet points signal unordered collections. (Prompt Engineering Guide) Code blocks should be used sparingly for non-code content, as OpenAI's own meta-prompt guidelines explicitly state "DO NOT USE `` CODE BLOCKS UNLESS SPECIFICALLY REQUESTED." (OpenAI)

For your architecture: Replace pure Markdown structure in (systemprompt.txt) with XML section tags. Keep Markdown inside sections for readability and token efficiency.

2. Agent frameworks converge on minimal system prompts with external state

A striking pattern emerges from analyzing 12 major agent frameworks: **the most successful modern frameworks use shorter system prompts** and push complexity into architecture, conversation patterns, or external files.

Framework	System Prompt Size	Core Pattern	Memory Strategy
AutoGPT	~800–1,500 tokens	Monolithic JSON-enforced	Vector DB + files
CrewAI	~200–500 tokens	Role-Goal-Backstory	Agent memory flag + YAML config
LangGraph	~50–200 tokens	Minimal + state graph	Checkpointers + injected store
SWE-Agent	~500–800 tokens	YAML config templates	Environment state injection
OpenHands	~500–1,000 tokens	Event-stream + tags	Sliding context + skills files
AutoGen	~10–50 tokens	Conversational	Chat history + LLM summarization
Claude Code	~15,000+ tokens (assembled)	Dynamic assembly from 110+ strings	Conversation summarization + CLAUDE.md
Cursor	~3,000–4,000 tokens	Persona + tool catalog	IDE state injection
Cline	~5,000+ tokens	XML tool format per action	Workspace snapshots, Plan/Act modes
Aider	Variable per model	Multiple edit formats	Repo map + chat summarization

The outlier is Claude Code, which assembles **110+ prompt strings** dynamically, totaling 15,000+ tokens — but it runs exclusively on Claude and can afford this because of prompt caching. (GitHub) For model-agnostic systems paying per-token every call, the LangGraph/SWE-Agent approach is more relevant: a lean core identity (~200–500 tokens) with state injected from external files.

Key universal patterns across all frameworks: the ReAct (Think → Act → Observe) cycle appears in nearly every framework; (Langchain) config-driven prompts (externalized in YAML or files) enable iteration without code changes; self-criticism or reflection mechanisms are built into many prompt structures; and "never output raw code — use tools instead" appears in Claude Code, Cursor, and Windsurf identically.

The most relevant architectural patterns for your system:

- **SWE-Agent's multi-template YAML config** — separating system template, instance template, next-step template, and error template (Princeton-nlp) mirrors your phase directive approach
- **Cline's model-agnostic XML tool format** — uses XML tags for tool invocation that works across any LLM backend, (GitHub) with configurable "prompt variants" for different model capabilities (DataCamp)
- **Aider's per-model adaptation** — tracks (examples_as_sys_msg), (reasoning_effort), and (thinking_tokens) per model, adjusting prompt strategy accordingly (GitHub)
- **Claude Code's sub-agent Task tool** — delegates complex operations to spawned sub-agents with their own scoped prompts (GitHub)

3. Persona drift is architectural, not just a prompting problem

The most concerning finding for long-running agents: **persona stability degrades >30% after 8–12 conversation turns**, (Medium) measured across LLaMA2-chat-70B and GPT-3.5-turbo-16k. The mechanism is attention decay — as conversation grows, the proportion of attention allocated to system prompt tokens drops sharply. (arXiv) Counterintuitively, **larger models experience greater identity drift** than smaller ones, measured across 9 LLMs including GPT family, LLaMA, Mixtral, and Qwen in 36-turn conversations. (arXiv)

The "Lost in the Middle" phenomenon (Liu et al., 2024, published in TACL) compounds this: **performance degrades by >30% when relevant information sits in the middle** of the context window. (arXiv) Models exhibit a U-shaped attention curve — they attend strongly to the beginning and end of context, with a trough in the middle. (AI Models) (arXiv) Serial position effects confirmed across all model families show that primacy effects are dominant in most scenarios. (arXiv) (Techxplore)

The practical implication for your architecture: your (workspace.md) injection strategy is exactly right, but its positioning matters enormously. Injecting it as a "fake tool-call result" near the end of context exploits recency bias. (Agent Prompt Optimization Re...) Your system prompt at the beginning exploits primacy bias. The instruction hierarchy ((system prompt > todo list > workspace.md > instructions.md > tool results)) should be explicitly stated in the prompt itself, (Agent Prompt Optimization Re...) because **the model cannot infer priority ordering from position alone** when the middle zone is attention-starved.

The most effective mitigation is **system prompt repetition** — periodically re-injecting key identity markers. Research confirms this excels at preventing drift in conversations with many turns. However, it consumes substantial context window space. (arXiv) The efficient compromise: repeat only the **3–5 most critical identity constraints** (not the full prompt) at the end of context, inside your workspace injection.

4. Reasoning directives must be conditional or absent entirely

The largest meta-analysis on chain-of-thought prompting — covering **110 papers with 1,218 experimental comparisons** — found that **CoT provides strong benefits primarily on math and symbolic reasoning tasks**, with near-zero or negative median improvements on commonsense, knowledge, and NLI tasks. (arXiv) A Wharton study on PhD-level questions found reasoning models (o3-mini, o4-mini, Gemini Flash 2.5) gained only +2–3% from external CoT while taking 20–80% longer. (upenn) (Upenn) Most alarmingly, one study measured a **36.3% absolute performance drop** for o1-preview compared to GPT-4o zero-shot on implicit statistical learning tasks when CoT was forced. (arXiv)

Each provider handles reasoning models differently:

- **OpenAI o-series:** Use (developer) role instead of (system). (OpenAI) CoT prompting "may not enhance performance and can sometimes hinder it." (CometAPI) Markdown is off by default (OpenAI) — include "Formatting re-enabled" to get it back. (Robertodiasduarte) (OpenAI) Zero-shot outperforms few-shot. Treat like a "senior coworker" — give goals, not procedures. (OpenAI) (CometAPI)
- **DeepSeek R1:** "Avoid adding a system prompt; all instructions should be contained within the user prompt." (GitHub) Few-shot "consistently degrades performance." (Together AI) (Helicone) Enforce (<think>\n at response start to prevent reasoning bypass. (Hugging Face)

- **Claude extended thinking:** "Remove all instructions about how to think." (DreamHost) The word "think" itself triggers specific behavior when extended thinking is disabled — replace with "consider," "believe," or "evaluate." (Anthropic) Adaptive thinking (Claude 4.6) dynamically decides when and how much to think. (Anthropic)

For your architecture: Your (reasoning_level) placeholder is the right instinct but wrong approach if it includes CoT instructions. The reasoning level should control API-level parameters ((reasoning_effort) for OpenAI, (budget_tokens) for Anthropic, temperature for DeepSeek) rather than adding textual directives. (Agent Prompt Optimization Re...) If you must include textual reasoning guidance, make it conditional: include it only for non-reasoning models, and strip it for o-series/R1/extended-thinking calls. A model-detection layer that adjusts prompt assembly based on model family is essential.

5. Your ~100-line prompt is near optimal — but position everything strategically

Research on prompt length reveals a tension: **reasoning performance degrades at ~3,000 tokens of input**, (Grit Daily) yet **longer domain-specific prompts generally improve task performance** compared to bare prompts. (arXiv) The resolution is signal-to-noise ratio, not raw length. A well-structured 16K-token prompt with RAG outperformed a monolithic 128K-token prompt in both accuracy and relevance. (PromptLayer)

Your current ~100 lines (~1,500–2,000 tokens for system + persona + phase directive) is in the sweet spot. The critical optimization is **positional**: most important instructions at the very beginning (first 20% of the prompt), contextual/domain information in the middle, and key constraints re-echoed near the end. (Teapot123) Your instruction hierarchy should be the literal first lines of the system prompt, not buried after identity setup.

Specific compression opportunities: your strategic phase directive at ~58 lines is likely too long relative to its importance-per-token. SWE-Agent achieves effective strategic behavior in ~15 lines of YAML config. Your persona.txt principles can be compressed from narrative to a numbered constraint list — models follow enumerated rules more reliably than prose descriptions. (Prompt Engineering Guide)

6. File-based memory is the dominant pattern — but make it structured

The AGENTS.md / CLAUDE.md pattern has seen explosive adoption across 40,000+ GitHub repositories. (Tessl) The architecture: persistent markdown files that are automatically injected into every conversation, creating self-improving knowledge bases. Analysis shows that "patterns that survive aren't clever prompts — they're structured memories that accumulate learning across runs." (Tessl)

Your (workspace.md) + (plan.md) + (todos.yaml) + (archive/) architecture aligns with best practice. The key refinement from research: implement a **"Memory Pinning" extraction step**. When the agent reads (instructions.md) in turn 1 and context compaction occurs in turn 20, those file contents (appearing as tool output) get lost. The solution: the first strategic phase must include a mandatory extraction step — "Upon reading instructions.md, extract Core Constraints, Style Guidelines, and Output Formats and write them permanently into workspace.md under a (## Pinned Instructions) header." This migrates instructions from transient context (tool output) to persistent context (workspace injection), surviving infinite compaction cycles.

(Agent Prompt Optimization Re...)

MemGPT's architecture provides the strongest theoretical foundation: an OS-inspired two-tier memory where "main context" (your workspace.md) acts as RAM and "archival storage" (your archive/) acts as disk.

Emergent Mind When tokens exceed ~70% of context window, a "memory pressure" warning triggers the agent to save important information. At 100%, queue eviction generates a recursive summary. **arXiv** Your architecture should implement this pressure threshold pattern rather than waiting for hard context limits.

7. Summarization prompts should produce structured JSON, not narrative

Your current summarization prompt producing structured JSON with `summary`, `tasks_completed`, `key_decisions`, `current_state`, `blockers`, `state_changes`, and `pinned_instructions` fields is well-designed. Research confirms structured summarization preserves specific entities (file paths, library versions, error codes) that narrative summarization loses. **Agent Prompt Optimization Re...** The critical addition: **your summarization prompt must explicitly instruct "Do NOT summarize the content of workspace.md or plan.md"** — those are persistent files that will be re-injected. Only summarize actions taken on those files, decisions made, and conversation dynamics. This prevents recursive summarization loss where each compaction degrades the representation of persistent state.

Comparative analysis of prompt architecture approaches

Design Dimension	Your Current Approach	Best Practice (Evidence-Based)	Gap
Structural format	Markdown only	XML sections + Markdown content	Migrate to hybrid
Instruction hierarchy	Stated in prompt	Stated first, re-echoed in workspace injection	Add re-echo at end
Identity persistence	workspace.md injection	+ pinned instruction extraction + identity re-echo	Add memory pinning
Reasoning directives	reasoning_level placeholder	API-level control + model-conditional assembly	Move to API layer
Phase directives	~58 lines strategic, ~27 tactical	~15–20 lines each, XML-delimited	Compress significantly
Summarization	Structured JSON (good)	+ explicit exclusion of persistent files	Add exclusion rule
Persona	Generalist remote worker narrative	Numbered constraint list with role + goal + backstory	Restructure for clarity
Context pressure	Not explicit	MemGPT-style thresholds at 70%/90%	Add pressure warnings
Tool instructions	In system prompt	Separate tool catalog, injected conditionally	Externalize tools

Design Dimension	Your Current Approach	Best Practice (Evidence-Based)	Gap
Model adaptation	reasoning_level variable	Full model-family detection with prompt variant assembly	Implement model profiles

Concrete recommendations for each prompt file

systemprompt.txt — restructure around XML sections and positional strategy

Keep: Placeholder architecture for dynamic assembly, instruction hierarchy declaration, agent name and expert identity injection, workspace.md injection point.

Change: Move instruction hierarchy to the literal first lines of the prompt. Wrap all sections in XML tags. Compress total base template from ~30 lines to ~25 lines. Add identity re-echo slot at the end (after workspace injection). Add explicit model-type conditional blocks.

Add: Memory pressure directive ("When context exceeds 70%, proactively consolidate volatile state into workspace.md"). Emergency consolidation trigger ("When context exceeds 90%, immediately stop current task, consolidate all state, and resume in next turn"). Explicit statement: "If a fact is not in workspace.md, it does not exist."

persona.txt — convert from narrative to structured constraint list

Keep: Generalist identity, principles about evidence-grounding and iterative delivery.

Change: Convert from prose narrative to numbered constraint list. Adopt CrewAI's Role-Goal-Backstory pattern as the opening structure. `CodeSignal` Compress from narrative principles to 8–10 crisp rules.

Add: Anti-drift anchors — 3 identity statements that should survive any summarization ("You are always [name]. You always [core behavior]. You never [boundary]."). Self-correction trigger: "If you notice yourself deviating from these constraints, pause and re-read this section."

strategic.txt — compress to essential decision framework

Change: Reduce from ~58 lines to ~20 lines. Remove procedural detail (git commands, specific file operations) and move those to workspace.md or a tools reference file. Focus on the decision framework: What to review, what questions to ask, what criteria for adapting the plan.

Add: Memory pinning directive: "During strategic phase, extract any new constraints discovered from file reads and pin them to workspace.md under `## Pinned Instructions`."

tactical.txt — tighten the tunnel-vision scope

Change: Keep the ~27-line length but restructure for positional emphasis. Lead with the single most important constraint (stay on current task, do not deviate). End with the completion criteria.

Add: Explicit tool-calling guidance: "Read before edit. Verify before commit. Test before declaring complete."

summarization_prompt.txt — add exclusion rules and identity preservation

Keep: The structured JSON schema with its current fields.

Change: Add explicit exclusion: "Do NOT summarize the content of workspace.md or plan.md — only summarize actions taken on them." Add identity preservation: "The `pinned_instructions` field MUST include the agent's role, current task description, and any active constraints — these survive compaction."

Add: A `context_pressure` field in the JSON output indicating approximate token usage as a percentage, triggering proactive consolidation in subsequent turns.

Draft improved prompts

Draft systemprompt.txt (~28 lines, ~400 tokens)

```
<instruction_hierarchy>
Priority: system prompt > phase directive > workspace.md > plan.md > instructions.md > tool results > conversation
history.
When instructions conflict, higher priority wins unconditionally.
</instruction_hierarchy>

<identity>
You are {agent_name}, {expert_identity}.
You operate in a managed workspace with persistent files.
You use tools to accomplish tasks — never output raw results when a tool action is appropriate.
If a fact is not in workspace.md, it does not exist for planning purposes.
</identity>

<phase_directive>
{prompt_content}
</phase_directive>

<workspace_state>
{workspace_md_content}
</workspace_state>

<active_tasks>
{todos_yaml_content}
</active_tasks>

<constraints>
- When context feels large, proactively consolidate volatile state into workspace.md.
- Extract constraints from file reads and pin them to workspace.md under ## Pinned Instructions.
- Read files before editing. Verify results before proceeding. Deliver what was asked.
```

- Distinguish between what you planned and what you actually executed.

</constraints>

<identity_anchor>

You are {agent_name}. Your current task: {current_task_summary}. Your active constraints: {pinned_constraints}.

</identity_anchor>

Design notes: The `<identity>` section exploits primacy bias by appearing early. The `<identity_anchor>` at the very end exploits recency bias — this is the critical anti-drift mechanism. The `<instruction_hierarchy>` is the literal first section because it governs how the model interprets everything that follows. Dynamic content (`{workspace_md_content}`), (`{todos_yaml_content}`) sits in the middle zone where attention is weakest, but these are reference data, not behavioral instructions. XML tags provide clear section boundaries that survive all model families. `(Anthropic)` Content within sections uses plain text and Markdown lists for token efficiency.

Draft persona.txt (~12 lines, ~200 tokens)

<role>

Generalist remote worker and expert problem-solver.

</role>

<goal>

Complete assigned tasks with high quality, grounded in evidence, delivering exactly what was requested.

</goal>

<backstory>

Experienced professional who understands complex tasks before acting, writes early and iterates, keeps the workspace concise and current, and always distinguishes planned actions from completed actions.

</backstory>

<operating_rules>

1. Understand the full task before beginning execution.
2. Ground every claim and decision in evidence from files, tools, or research.
3. Write early drafts and iterate — do not wait for perfection.
4. Keep workspace.md concise; remove stale information, update with current state.
5. Deliver what was asked, in the format requested, at the quality expected.
6. When uncertain, state uncertainty explicitly rather than fabricating confidence.
7. Never fabricate file contents, tool outputs, or research findings.
8. If you notice yourself deviating from these rules, pause and re-read this section.

</operating_rules>

Draft strategic.txt (~20 lines, ~300 tokens)

<strategic_phase>

You are in STRATEGIC mode. Purpose: review progress, identify problems, adapt the plan.

Review protocol:

1. Read workspace.md and plan.md for current state of truth.
2. Check recent tool results and conversation for new information.
3. Evaluate: Is the current plan still valid? What has changed?
4. Identify blockers, completed tasks, and next priorities.
5. Update plan.md with revised strategy if needed.
6. Update workspace.md with any new state.

Memory maintenance:

- Extract any new constraints discovered from file reads and pin them to workspace.md under ## Pinned Instructions.
- Remove stale entries from workspace.md that no longer reflect reality.
- If workspace.md exceeds 50 lines, consolidate aggressively.

Decision criteria for adapting the plan:

- Evidence of a blocked path warrants pivoting, not persisting.
- Completion of a milestone warrants reviewing the next milestone's assumptions.
- Discovery of new requirements warrants updating the task list before continuing.

When strategic review is complete, transition to tactical phase with a clear next action.

</strategic_phase>

Draft tactical.txt (~15 lines, ~200 tokens)

<tactical_phase>

You are in TACTICAL mode. Purpose: execute the current task with focused, sequential action.

Primary constraint: Stay on the current task. Do not start new tasks, reorganize the plan, or explore tangents.

Execution protocol:

1. Identify the single next action from your task list.
2. Read any files you need before modifying them.
3. Execute the action using the appropriate tool.
4. Verify the result — check for errors, confirm expected output.
5. Update workspace.md with what you actually did (not what you planned).
6. Proceed to the next action, or signal completion if the task is done.

Completion criteria: The task is complete when its acceptance criteria are met and the result is verified. If you cannot verify, state what verification is needed.

</tactical_phase>

Draft summarization_prompt.txt (~30 lines, ~400 tokens)

Produce a structured JSON summary of the conversation since the last summary.

Output raw JSON only — no markdown formatting, no code blocks, no commentary.

EXCLUSIONS: Do NOT summarize the content of workspace.md or plan.md. These files are persistent and will be re-injected. Only summarize ACTIONS TAKEN on these files (e.g., "Updated workspace.md with new blockers").

Required JSON schema:

```
{
  "summary": "Concise narrative of what happened (max 3 sentences)",
  "tasks_completed": ["List of tasks finished with outcomes"],
  "tasks_in_progress": ["List of tasks started but not finished"],
  "key_decisions": ["Architectural or strategic choices made, with reasoning"],
  "current_state": "One sentence describing where the agent is right now",
  "blockers": ["Active blockers or errors preventing progress"],
  "state_changes": ["Files created, modified, or deleted"],
  "pinned_instructions": ["Active constraints extracted from files that must persist"],
  "identity_anchor": {
    "agent_role": "Current role/persona",
    "current_task": "What the agent is working on right now",
    "active_constraints": ["Top 3-5 constraints that must survive compaction"]
  },
  "context_pressure": "estimated percentage of context window used"
}
```

Preserve specifics: file paths, exact error messages, version numbers, and quantitative data.
Generic summaries like "made progress on the task" are failures — be precise about WHAT happened.

The five most actionable changes, ranked by impact

First, migrate from pure Markdown structure to XML section tags with Markdown content. This single change improves cross-model compatibility more than any other formatting decision. Every major provider supports this pattern, [Robertodiasduarte](#) and it provides cleaner boundaries between dynamic and static content.

Second, implement the identity anchor pattern — repeat the 3 most critical identity facts (agent name, current task, active constraints) at the end of the system prompt, after workspace injection. This exploits the U-shaped attention curve where models attend most strongly to the beginning and end of context. [Stanford University](#) Research shows >30% performance degradation for information positioned in the middle. [arXiv](#) [Maxim Articles](#)

Third, add a model-type detection layer that adjusts prompt assembly. For reasoning models (o-series, R1, Claude extended thinking): strip all CoT instructions, reduce prompt verbosity, use API-level reasoning parameters instead of textual directives. [Helicone](#) [OpenAI](#) For standard models: include the full prompt with optional few-shot examples. [Anthropic](#) [OpenAI](#) Aider's per-model profile system is the closest production reference for this pattern.

Fourth, implement memory pinning with explicit extraction directives. The first strategic phase after reading any instruction file must extract and pin constraints to workspace.md. This prevents the "read-once" problem where file contents vanish after context compaction. [Agent Prompt Optimization Re...](#)

Fifth, add context pressure thresholds. At 70% of the context window, inject a warning that triggers proactive consolidation. At 90%, force an emergency consolidation phase that strips tools and compacts all volatile state immediately. (Medium) MemGPT demonstrates this pattern can reduce token usage by ~90% on long-context benchmarks while preserving task-critical information. (Emergent Mind)